

Data Warehouse

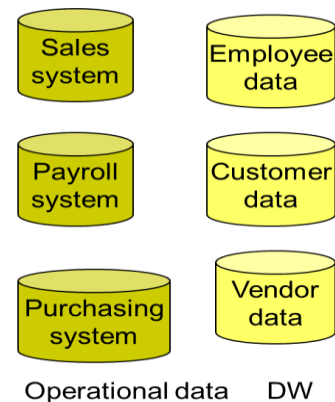
In most of the organization, there occur large database in operation for normal daily transactions called operational database. A data warehouse is a large database built from the operational database that organizes all the data available in an organization, makes it accessible & usable for the all kinds of data analysis and also allows to create a lots of reports by the use of mining tools.

“A data warehouse is a subject-oriented, integrated, time-variant, and non-volatile collection of data in support of management’s decision-making process.”

A data warehouse should be:

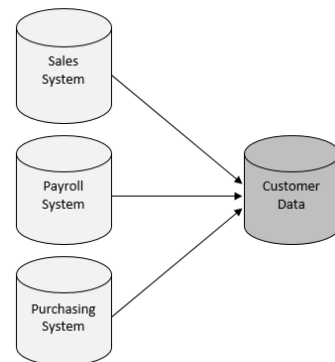
Subject oriented:

- Focus is on Subject Areas rather than Applications
- Organized around major subjects, such as customer, product, sales.
- Provide a simple and concise view around particular subject issues by excluding data that are not useful in the decision support process.



Integrated

- Constructed by integrating multiple, heterogeneous data sources. The result is that data-once it resides in the data warehouse- has a single physical corporate appearance.
- Integration tasks handles naming conventions as well as physical attributes of data.



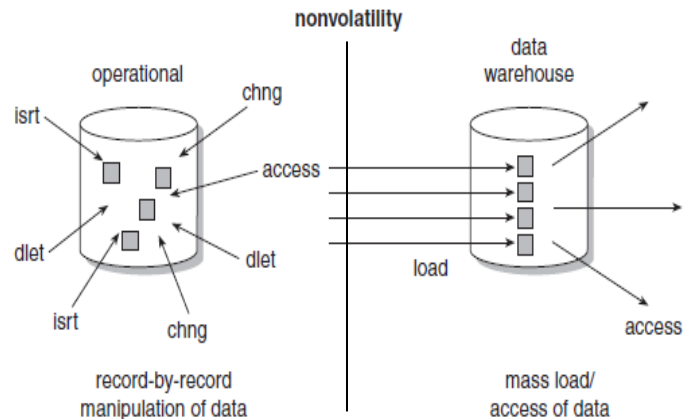
Time variant

- The time horizon for the data warehouse is significantly longer than that of operational systems.
 - Operational database: current value data. (60 to 90 days)

- Data warehouse data: provide information from a historical perspective (e.g., past 5-10 years)

Non volatile

- Data Warehouse is relatively static in nature.
- Data warehouse data is loaded (usually en masse) and accessed, but it is not updated (in the general sense).
- Data warehouse data is a sophisticated series of snapshots, each taken at one moment in time. The effect created by the series of snapshots is that the data warehouse has a historical sequence of activities and events.



Architecture of a Data Warehouse System

A typical data warehouse system has three main phases:

- **Data acquisition**
 - Relevant data collection
 - Recovering: transformation into the data warehouse model from existing models
 - Loading: cleaning and loading in the DW
- **Storage**
- **Data extraction**

Tool examples: Query report, SQL, multidimensional analysis (OLAP tools), data mining

These three tasks are performed by following personnel:

Load Manager: The system components that perform all the operations necessary to support the extract and load process. It fast loads the extracted data into a temporary data store and

performs simple transformations into a structure similar to the one in the data warehouse. Also called ETL (Extract Transform and Load).

Warehouse Manager (Data Manager): It is the system component that performs analysis of data to ensure consistency. The data from various sources and temporary storage are merged into data warehouse by the warehouse manager. The job of backing-up and archiving data as well as creation of index is performed by this manager.

Query Manager: Performs all the operations necessary to support the query management process by directing queries to the appropriate tables. In some cases it also stores query profiles to allow the warehouse manager to determine which indexes and aggregations are

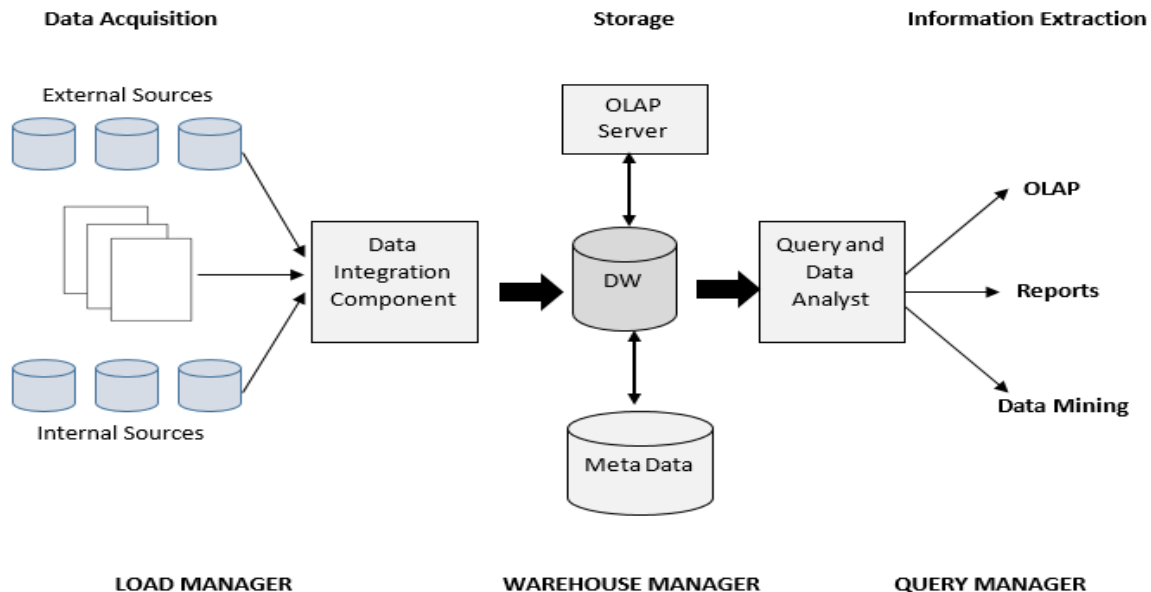


Figure. Data Warehouse Architecture

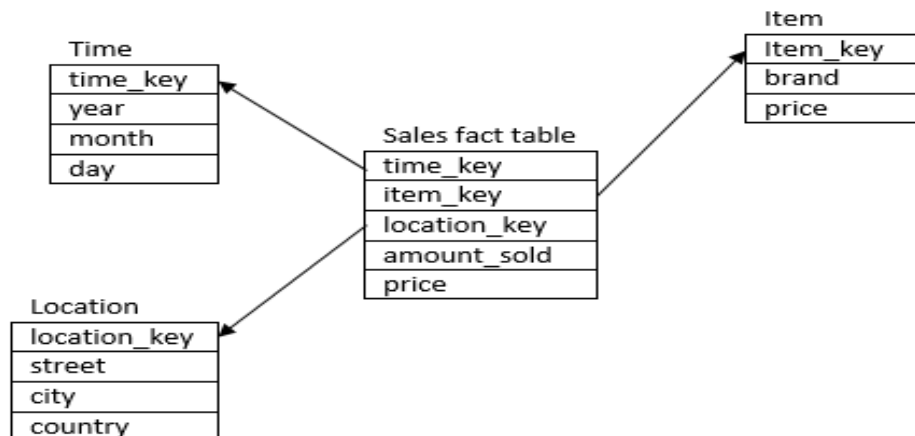
appropriate.

2.1OLAP

Schemas for Multidimensional Databases: **Stars, Snowflakes, and Fact Constellations**

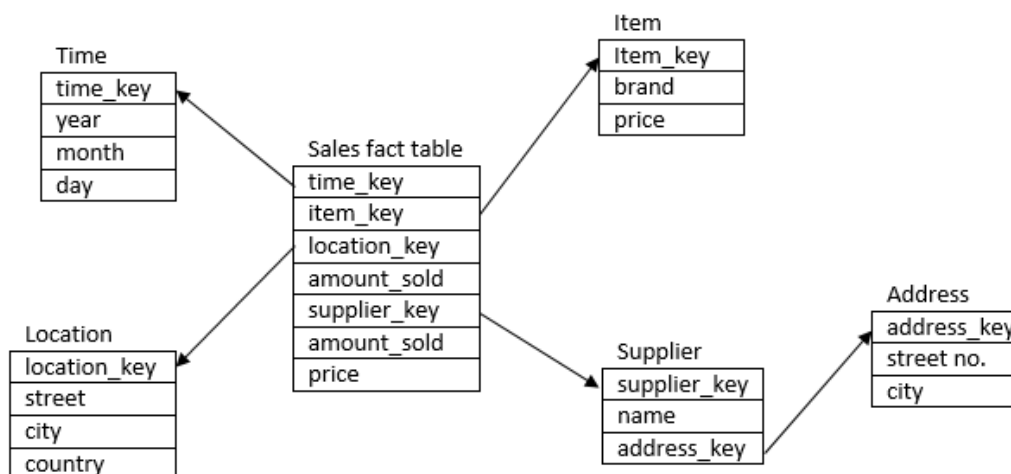
a. Star Schema

The most common modeling paradigm is the star schema, in which the data warehouse contains a large central table (fact table) containing the bulk of the data with no redundancy, and a set of smaller attendant tables (dimension tables), one for each dimension. The schema graph resembles a starburst, with the dimension tables displayed in a radial pattern around the central fact table.



b. Snowflake Schema

The snowflake schema is a variant of the star schema model, where some dimension tables are *normalized*, thereby further splitting the data into additional tables. The resulting schema graph forms a shape similar to a snowflake.

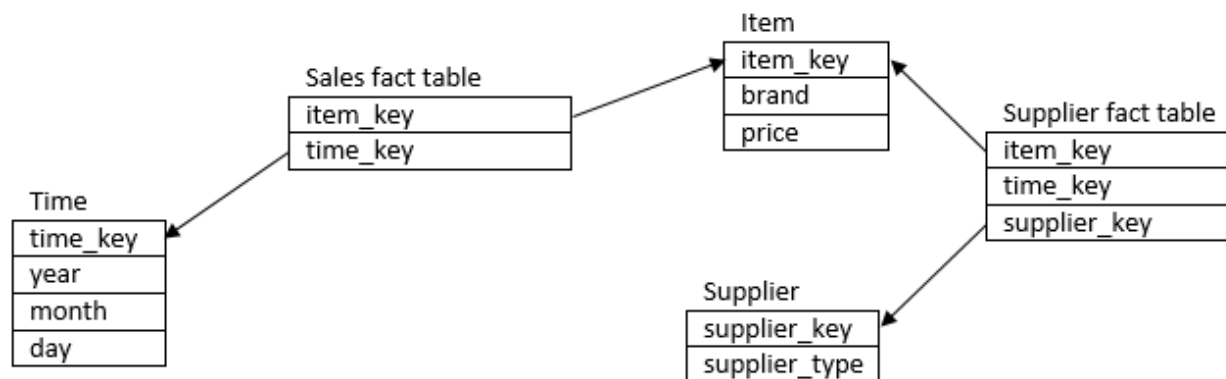


The major difference between the snowflake and star schema models is that the dimension tables of the snowflake model may be kept in normalized form to reduce redundancies. Such a table is easy to maintain and saves storage space. However, this saving of space is negligible

in comparison to the typical magnitude of the fact table. Furthermore, the snowflake structure can reduce the effectiveness of browsing, since more joins will be needed to execute a query. Consequently, the system performance may be adversely impacted. Hence, although the snowflake schema reduces redundancy, it is not as popular as the star schema in data warehouse design.

c. Fact constellation schema

Sophisticated applications may require multiple fact tables to *share* dimension tables. This kind of schema can be viewed as a collection of stars, and hence is called a galaxy schema or a fact constellation.



OLAP

OLAP (online analytical processing) is a term used to describe the analysis of complex data from the data warehouse. The analysis often requires resource intensive aggregations processing and therefore it becomes necessary to implement a special database (e.g. data warehouse) to improve

OLAP response time. In the hands of skilled knowledge workers, OLAP tools use distributed computing capabilities for analyses that require more storage and processing power than can be economically and efficiently located on an individual desktop. OLAP has two immediate consequences: online part requires the answers of queries to be fast, and the analytical part is a hint that the queries itself are complex. So, OLAP are **Complex questions with Fast Answers**.

Definition: *OLAP is the dynamic enterprise analysis required to create, manipulate, animate and synthesize information from exegetical, contemplative and formulaic data analysis models.*

Essentially what this definition means is that the information is manipulated from the point of view of a manager (exegetical), from the point of view of someone who has thought about it (contemplative) and according to some formula (formulaic).

An even simpler definition is that OLAP is a fast analysis of shared multidimensional information for advanced analysis. Or FASMI.

Types of OLAP

i Relational OLAP (ROLAP)

These are the intermediate servers that stand in between a relational back-end server and client front-end tools. It provides functionality by using relational databases and relational query tools to store and analyze multidimensional data. This model permits multidimensional analysis of data as this enables users to perform a function equivalent to that of the traditional OLAP slicing and dicing feature.

One advantage of ROLAP over the other styles of OLAP analytic tools is that it is deemed to be more scalable in handling huge amounts of data. ROLAP sits on top of relational databases therefore enabling it to leverage several functionalities that a relational database is capable of. Another gain of a ROLAP tool is that it is efficient in managing both numeric and textual data. It also permits users to “drill down” to the leaf details or the lowest level of a hierarchy structure. However, ROLAP applications display a slower performance as compared to other style of OLAP tools since, oftentimes, calculations are performed inside the server. Another demerit of a ROLAP tool is that as it is dependent on use of SQL for data manipulation, it may not be ideal for performance of some calculations that are not easily translatable into an SQL query.

ii Multidimensional OLAP (MOLAP)

MOLAP uses array-based multidimensional storage engines for multidimensional views of data. With multidimensional data stores, the storage utilization may be low if the data set is sparse. One of the major distinctions of MOLAP against a ROLAP tool is that data are pre-summarized and are stored in an optimized format in a multidimensional cube, instead of in a relational database. This is OLAP tool is efficient to use in making analysis reports since this enables users to easily reorganize or rotate the cube structure to view different aspects of data. This is done by way of slicing and dicing. MOLAP analytic tool are also capable of performing complex calculations. Since calculations are predefined upon cube creation, this results in the faster return of computed data. MOLAP systems also provide users the ability to quickly write back data into a data set. Moreover, in comparison to ROLAP, MOLAP is considerably less heavy on hardware due to compression techniques. In a nutshell, MOLAP is more optimized for fast query performance and retrieval of summarized information.

There are certain limitations to implementation of a MOLAP system, one primary weakness of which is that MOLAP tool is less scalable than a ROLAP tool as the former is capable of handling only a limited amount of data. The MOLAP approach also introduces data redundancy. There are also certain MOLAP products that encounter difficulty in updating models with dimensions with very high cardinality.

iii Hybrid OLAP (HOLAP)

HOLAP is the product of the attempt to incorporate the best features of MOLAP and ROLAP into a single architecture. This tool tried to bridge the technology gap of both products by enabling access or use to both multidimensional database (MDDB) and Relational Database Management System (RDBMS) data stores. HOLAP systems stores larger quantities of detailed data in the relational tables while the aggregations are stored in the pre-calculated cubes. HOLAP also has the capacity to “drill through” from the cube down to the relational tables for delineated data.

Some of the advantages of this system are better scalability, quick data processing and flexibility in accessing of data sources.

OLTP

OLTP stands for OnLine Transactional Processing. OLTP involves short, simple, frequent queries and/or modifications, each involving a small number of tuples. For example in a banking system cover most of the day-to-day operations, such as purchasing, inventory, manufacturing, banking, payroll, registration, and accounting.

The differences between OLTP and OLAP are as follows:

1. Users: OLTP systems are designed for office workers while the OLAP systems are designed for Decision makers. Therefore while an OLTP system may be accessed by hundreds or even thousands of users in a large enterprise, an OLAP system is likely to be accessed only by a select group of managers and may be used only by dozens of users.

2. Functions: OLTP systems are mission-critical. They support day-to-day operations of an enterprise and are mostly performance and availability driven. These systems carry out simple repetitive operations. OLAP systems are management-critical to support decision of an enterprise support functions using analytical investigations. They are more functionality driven. These are often much more complex operations.

3. Nature: Although SQL queries often return a set of records, OLTP systems are designed to process one record at a time, for example a record related to the customer who might be on the phone or in the store. OLAP systems are not designed to deal with individual customer records. Instead they involve queries that deal with many records at a time and provide summary or aggregate data to a manager. OLAP applications involve data stored in a data warehouse that has been extracted from many tables and perhaps from more than one enterprise database.

4. Design: OLTP database systems are designed to be application-oriented while OLAP systems are designed to be subject-oriented. OLTP systems view the enterprise data as a collection of tables (perhaps based on an entity-relationship model). OLAP systems view enterprise information as multidimensional).

5. Data: OLTP systems normally deal only with the current status of information. For example, information about an employee who left three years ago may not be available on the Human Resources System. The old information may have been achieved on some type of

stable storage media and may not be accessible online. On the other hand, OLAP systems require historical data over several years since trends are often important in decision making.

6. Kind of use: OLTP systems are used for reading and writing operations while OLAP systems normally do not update the data.

parameters	OLTP	OLAP
Purpose	It is a system for processing large volumes of real-time transactional data.	It is a system for the multidimensional analysis of consolidated business data.
Usage	It is used for adding, deleting, or updating databases to keep the data up-to-date.	It is used to make business decisions through queries and complex analyses of large amounts of data.
Focus	The system is more focused on transactional data maintenance and less on data analysis.	The system is focused on data analysis and not on maintaining day-to-day transactions.
data source	OLTP sources data from traditional database management systems.	OLAP has multiple data sources, which include real-time and historical databases, including OLTP.
Data type	The data consists of a large number of short transactions.	The system processes large volumes of data from multiple sources.
Processing time	Very low processing time at the scale of a few milliseconds.	Depending on the query, processing time is not as fast as OLTP systems and may range from a few seconds to hours.
Query	Related to adding, deleting, and updating data.	Related to data analysis.
Availability	OLTP systems are available round-the-clock and updated frequently to maintain data integrity.	OLAP systems don't need to be updated so frequently since their functions are analytic in nature.
Normalization	Data tables are normalized.	Data tables are not normalized.
Backup	Requires constant backup and recovery.	Can be backed up less frequently.

user volume	Supports large user volume simultaneously.	Accommodates multiple users but doesn't have a large user volume like OLTP.
Operations	Allows both read and write operations.	Usually supports read-only operations.
process	Processes day-to-day data quickly.	Processes analytical queries consistently and at a fast pace.

2.2 Multidimensional view and data cube

“What is a data cube?” A data cube allows data to be modeled and viewed in multiple dimensions. It is defined by dimensions and facts.

In general terms, **dimensions** are the perspectives or entities with respect to which an organization wants to keep records. For example, an organization, *AllElectronics* may create a *sales* data warehouse in order to keep records of the store's sales with respect to the dimensions *time*, *item*, *branch*, and *location*. These dimensions allow the store to keep track of things like monthly sales of items and the branches and locations at which the items were sold. Each dimension may have a table associated with it, called a *dimension table*, which further describes the dimension. For example, a dimension table for *item* may contain the attributes *item_name*, *brand*, and *type*.

The multidimensional storage model involves two types of tables: dimension tables and fact tables. A **dimension table** consists of tuples of attributes of the dimension. **Fact table** is usually a tabular representation of the data. The **fact table** contains the names of the *facts*, or measures, as well as keys to each of the related dimension tables.

Examples of facts for a sales data warehouse include *dollars sold* (sales amount in dollars), *units sold* (number of units sold), and *amount budgeted*.

Although we usually think of cubes as 3-D geometric structures, in data warehousing the data cube is n -dimensional.

To understand what a 3-D data cube, let's start by looking at a simple 2-D data cube that is, in fact, a table or spreadsheet for sales data from *abcDrinks* enterprise. In particular, we will look at the *AbcDrinks* sales data for items sold per month in the city of Lalitpur. These data are shown in following table:

Location= Lalitpur				
		Items		
Time(quarter)		Coke	Fanta	Sprite
	Q1	15	20	10
	Q2	20	15	5
	Q3	15	10	20
	Q4	5	20	15

In this 2-D representation, the sales for Lalitpur are shown with respect to the *time* dimension (organized organized to quarters) and the *item* dimension (organized according to the product of items sold). The fact or measure displayed is *number of items*. Now, suppose that we would like to view the sales data with a third dimension. For instance, suppose we would like to view the data according to *time* and *item*, as well as *location* for the cities Kathmandu, Lalitpur, and Kavre. These 3-D data are shown in following table:

		Location= Lalitpur			Location= Kathmandu			Location=Kavre		
		Items			Items			Items		
Time(Quarter)		Coke	Fanta	Sprite	Coke	Fanta	Sprite	Coke	Fanta	Sprite
	Q1	15	20	10	7	10	20	5	20	10
	Q2	20	15	5	20	15	25	25	15	10
	Q3	15	10	20	3	10	20	7	5	5
	Q4	5	20	15	15	10	20	5	25	10

Conceptually, we may also represent the same data in the form of a 3-D data cube, as in following figure:

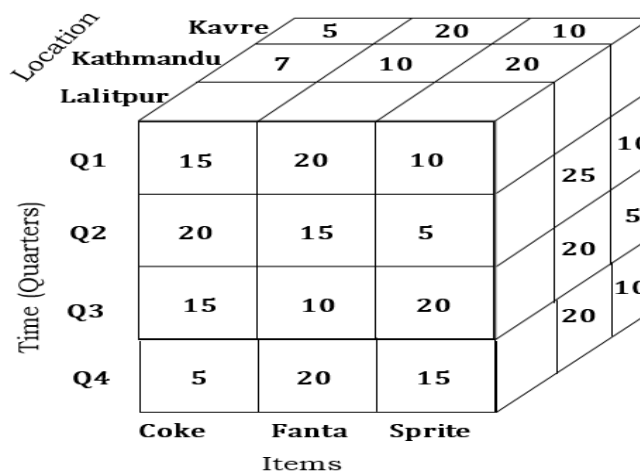


Figure. 3-D data cube representation of the data according to the dimensions time, items and location

In a cube with n dimensions, there are 2^n possible aggregation operations. So, in the three-dimensional cube, the following eight types of aggregations or queries are possible:

1. null [ALL, ALL, ALL]
e.g. How many items have been sold in total?
One possible query
2. time [a, ALL, ALL]
e.g. How many item have been sold in Q1?
Four possible query
3. items [ALL, b, ALL]
e.g. How many sprite have been sold?
three possible query
4. location [ALL, ALL, c]
e.g. How many items have been sold in Kathmandu?
three possible query
5. time, item [a, b, ALL]
e.g. How many sprite sold in Q4?
12 possible queries. ($4 \times 3 \times 1$).
6. Location, time [a, ALL, c]
e.g. How many items sold in Lalitpur in Q3?
12 possible queries
7. Item, location [ALL, b, c]
e.g. How many sprite sold in Lalitpur?
9 possible queries.
8. all [a, b, c]
e.g. How many Fanta sold in Kavre in Q3?
36 possible queries.

The aggregate operations can be represented in a lattice of cuboids. Given a set of dimensions, we can generate a cuboid for each of the possible subsets of the given dimensions. The result would form a *lattice* of cuboids, each showing the data at a different level of summarization, or group by. The lattice of cuboids is then referred to as a data cube.

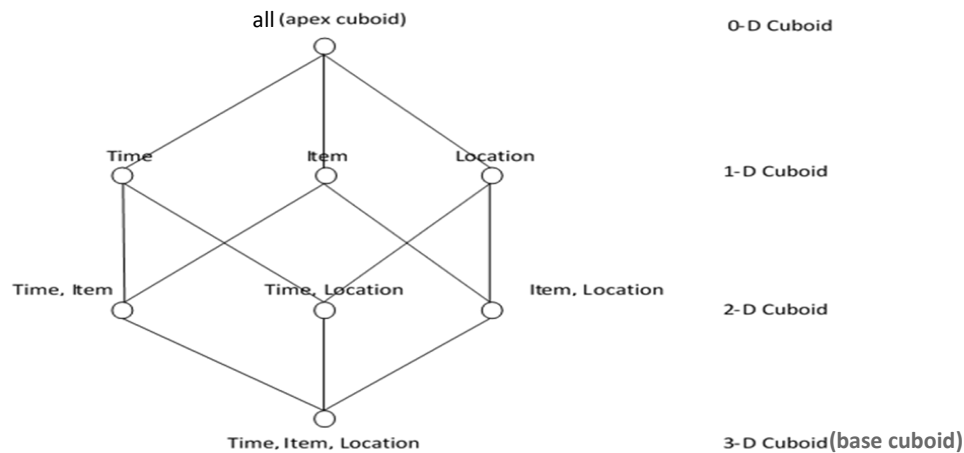


Figure. Lattice of cuboids, making up a 3-D data cube. Each cuboid represents a different group-by.

Figure above is a 3-D cuboid for *month*, *item*, and *location*, summarized for all suppliers. The 0-D cuboid, which holds the highest level of summarization, is called the apex cuboid. The cuboid that holds the lowest level of summarization is called the base cuboid. The apex cuboid is typically denoted by all.

Suppose that we would now like to view our sales data with an additional fourth dimension, such as *supplier*. Viewing things in 4-D becomes tricky. However, we can think of a 4-D cube as being a series of 3-D cubes, as shown in following figure. If we continue in this way, we may display any n -D data as a series of $(n-1)$ -D “cubes.” The data cube is a metaphor for multidimensional data storage.

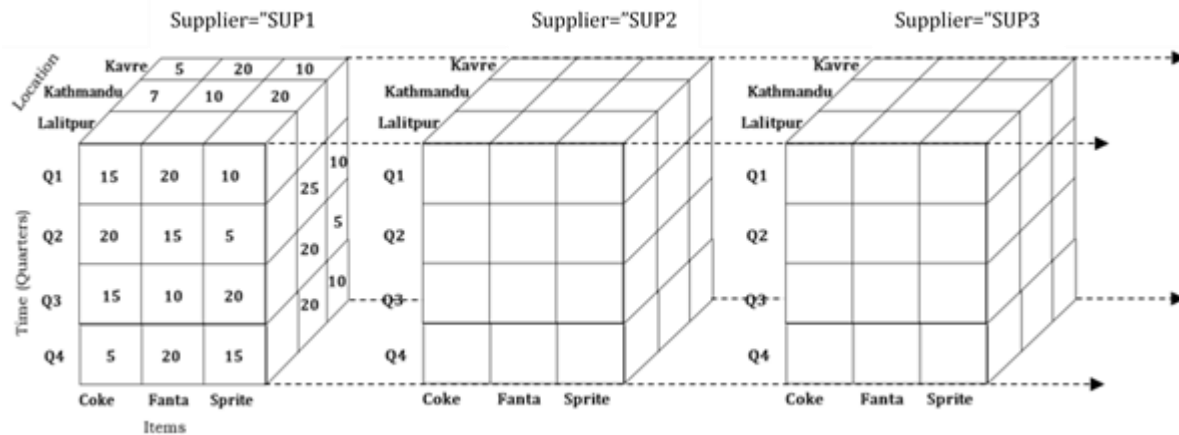


Figure. A 4-D data cube representation of sales data, according to the dimensions time, item, location and supplier

The important thing to remember is that data cubes are n -dimensional and do not confine data to 3-D.

Figure below shows a lattice of cuboids forming a data cube for the dimensions *month*, *item*, *location*, and *supplier*

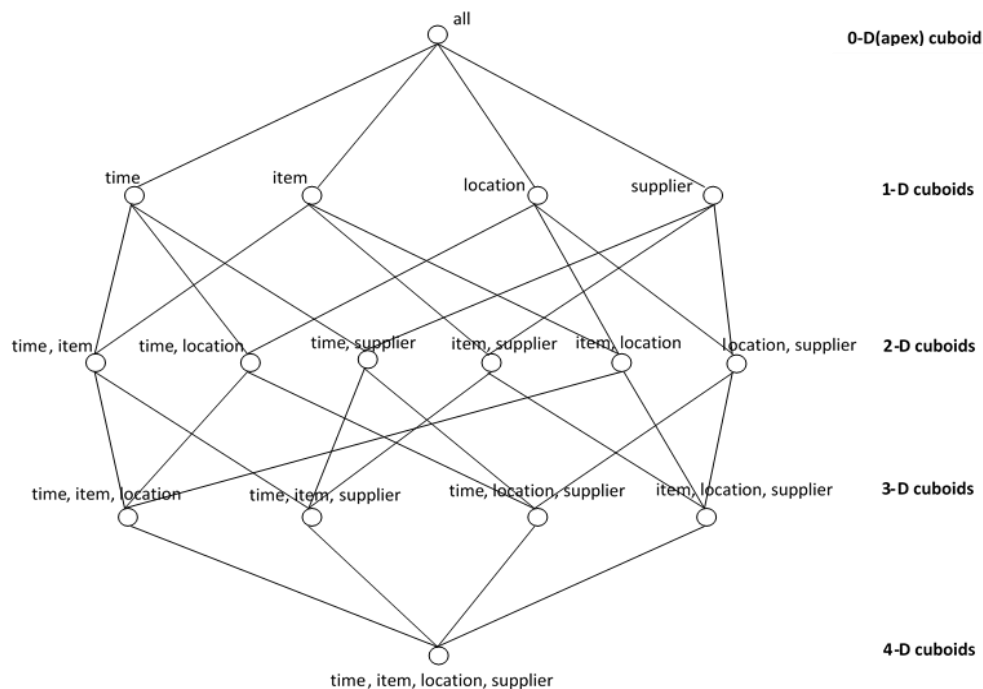


Figure. Lattice of cuboids, making up a 4-D data cube for the dimensions time, item, location, and supplier

2.3 Data Cube implementation

i. Full Materialization (Pre compute and store all)

This means that millions of aggregates will need to be computed and stored. Although this is the best solution as far as query response time is concerned, the solution is impractical since resources required to compute the aggregates and to store them will be prohibitively large for a large data cube.

ii. No Materialization (Pre compute none)

This means that the aggregates are computed on the fly using the raw data whenever a query is posed. This approach does not require additional space for storing the cube but the query response time is likely to be very poor for large data cubes.

iii. Partial Materialization (Pre compute and store some)

This means that we pre-compute and store the most frequently queried aggregates and compute others as the need arises. We may also be able to derive some of the remaining aggregates using the aggregates that have already been computed. It may therefore be worthwhile also to pre compute some aggregates that are not most frequently queried but help in deriving many other aggregates. It will of course not be possible to derive all the aggregates from the pre-computed aggregates and it will be necessary to access the database to compute the remaining aggregates. The more aggregates we are able to pre-compute the better the query performance.

2.4 Data Cube Operations

Hierarchies:

A concept hierarchy defines a sequence of mappings from a set of low-level concepts to higher-level, more general concepts. For example Street→City→State→Country or Day < Month < Quarter < Year. Climbing up a hierarchy means reducing the dimension.

“How are concept hierarchies useful in OLAP?” In the multidimensional model, data are organized into multiple dimensions, and each dimension contains multiple levels of abstraction defined by concept hierarchies. This organization provides users with the flexibility to view data from different perspectives.

OLAP Operations

i. Roll up

The roll-up operation (also called the *drill-up* operation by some vendors) performs aggregation on a data cube, either by *climbing up a concept hierarchy* for a dimension or by *dimension reduction*. Figure 5 shows the result of a roll-up operation performed on the central cube by climbing up the concept hierarchy for *location*.

ii. Drill down

Drill-down is the reverse of roll-up. It navigates from less detailed data to more detailed data. Drill-down can be realized by either *stepping down a concept hierarchy* for a dimension or *introducing additional dimensions*. Drill-down in figure 5 occurs by descending the *time* hierarchy from the level of *quarter* to the more detailed level of *month*.

iii. Slice

The *slice* operation performs a selection on one dimension of the given cube, resulting in a subcube. Figure below shows a slice operation where the sales data are selected from the central cube for the dimension *time* using the criterion *time* = “Q1”.

iv. Dice

The *dice* operation defines a subcube by performing a selection on two or more dimensions. Figure 5 shows a dice operation on the central cube based on the following selection criteria that involve three dimensions: (*location* = “Toronto” or “Vancouver”) and (*time* = “Q1” or “Q2”) and (*item* = “home entertainment” or “computer”).

v. Pivot (rotate):

Pivot (also called *rotate*) is a visualization operation that rotates the data axes in view in order to provide an alternative presentation of the data. Figure 5 shows a pivot operation where the *item* and *location* axes in a 2-D slice are rotated. We can also perform the rotation in 3-D cube.

OLAP can generate summarizations, aggregations, and hierarchies at each granularity level and at every dimension intersection. OLAP also supports functional models for forecasting, trend analysis, and statistical analysis. In this context, an OLAP engine is a powerful data analysis tool.

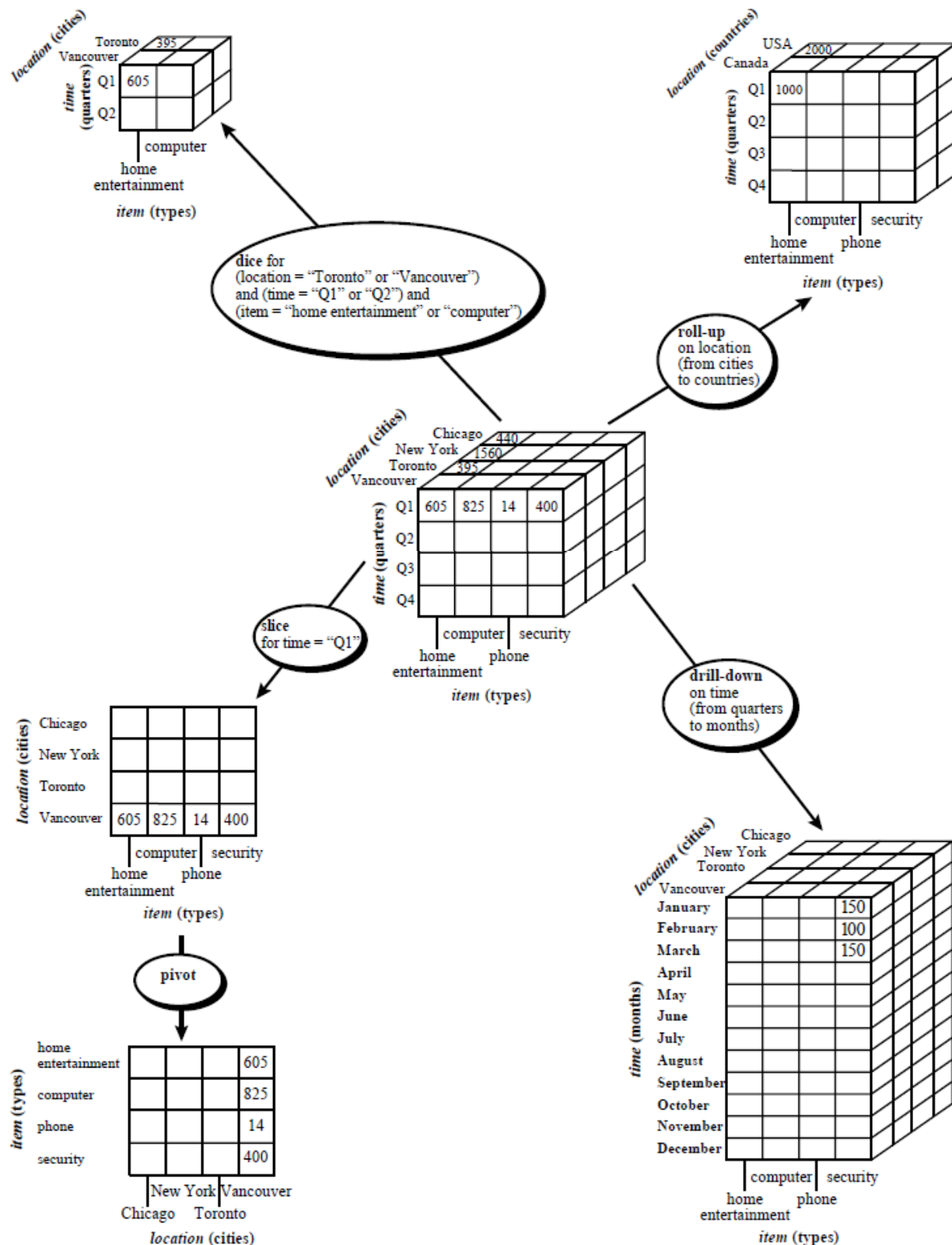


Figure 5 Examples of typical OLAP operations on multidimensional data.

References

Han, J., Pei, J., & Tong, H. (2023). *Data Mining Concept and Techniques*. Cambridge, MA: Elsevier Inc.

Inmon, W. (2002). *Building the Data Warehouse*. Wiley Computer Publishing.

Tang, P.-N., Steinbach, M., & Kumar, V. (2006). *Introduction to Data Mining*. Boston, MA: Pearson Education Inc.